



รายงานการออกแบบ REST API และ ไคลเอนต์ (Workout Log)

ผู้พัฒนา: รัชภูมิ เล่าคุ้ม | รหัสนักศึกษา: 660910873

1. บทสรุปและสถาปัตยกรรมระบบ (System Architecture)

โปรเจกต์ Workout Log ออกแบบขึ้นในรูปแบบ Single Page Application (SPA) โดยแยกการทำงานระหว่างฝั่งเซิร์ฟเวอร์ (Backend) และฝั่งไคลเอนต์ (Frontend) ออกจากกันอย่างชัดเจนตามหลักสถาปัตยกรรมซอฟต์แวร์

- `/server` : พัฒนาด้วย Node.js และ Express Framework ทำหน้าที่เป็น RESTful API สำหรับประมวลผลและจัดการข้อมูล
- `/public` : ประกอบด้วย HTML, CSS และ JavaScript ฝั่งไคลเอนต์ ทำหน้าที่จัดการแสดงผล UI และรับ-ส่งข้อมูลกับเซิร์ฟเวอร์ผ่านคำขอ Asynchronous Fetch API
- `/docs` : รวบรวมเอกสารประกอบการใช้งานและภาพถ่ายหลักฐานการดีบักระบบ

2. ข้อกำหนดการออกแบบ REST API (Endpoints)

ระบบให้บริการ API สำหรับจัดการข้อมูลรายการออกกำลังกายด้วยมาตรฐาน HTTP Verbs และ JSON Data Structure ดังนี้:

Method	Endpoint	คำอธิบาย	Request Body / Query	Status Code
GET	<code>/api/workouts</code>	ดึงรายการออกกำลังกายทั้งหมด	Query: <code>?category=...</code> (Optional)	200 OK
GET	<code>/api/workouts/:id</code>	ดึงข้อมูลรายการตาม	-	200 OK / 404 Not

Method	Endpoint	คำอธิบาย	Request Body / Query	Status Code
		ID		Found
POST	/api/workouts	เพิ่มรายการ ออกกำลังกาย ใหม่	JSON: { title, category, duration, date }	201 Created / 400 Bad Request
DELETE	/api/workouts/:id	ลบรายการ ออกกำลังกาย ตาม ID	-	204 No Content / 404 Not Found

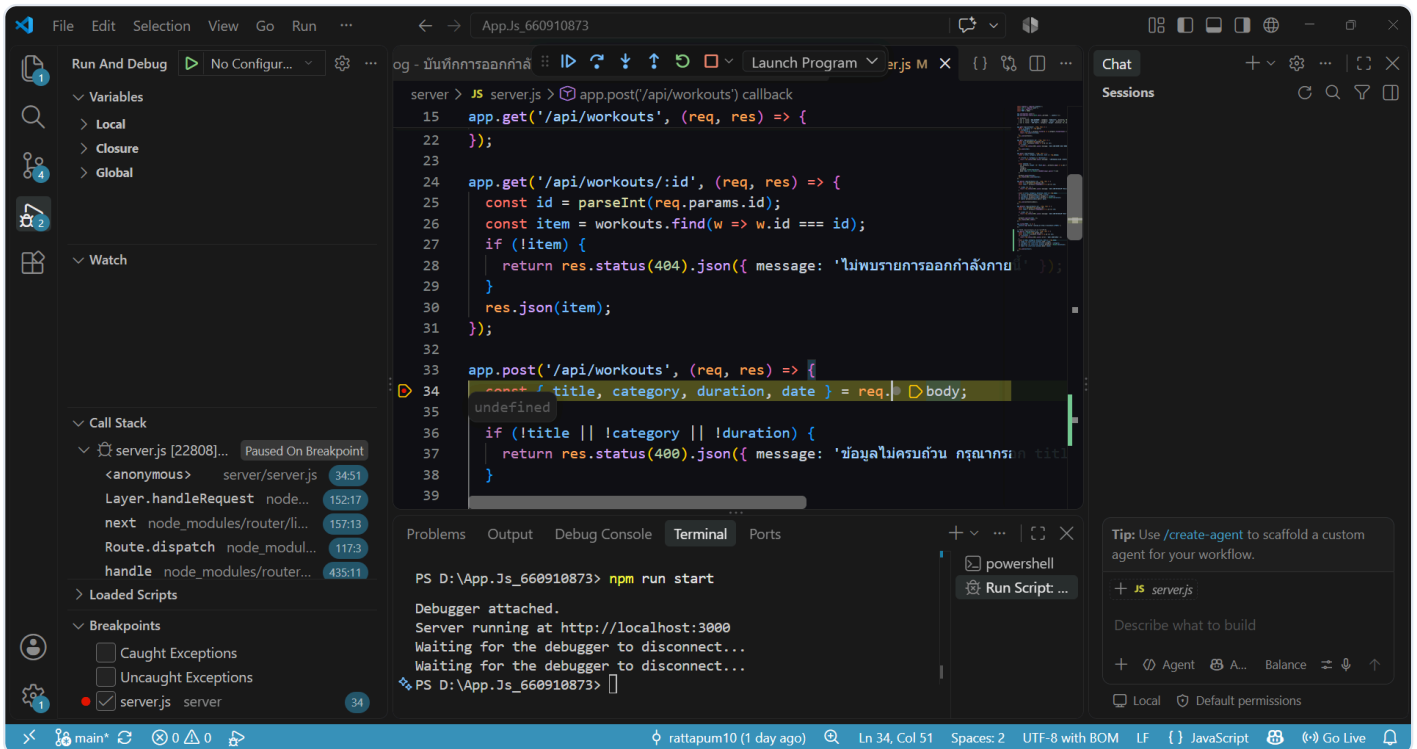
3. การทำงานฝั่งไคลเอนต์ (Client-Side Logic)

- การแสดงผลข้อมูล (GET):** ไคลเอนต์เรียกใช้ฟังก์ชัน `fetchWorkouts()` ส่งคำขอ `GET /api/workouts` จากนั้นนำข้อมูล JSON มาสร้างแท็ก DOM แสดงผลบนหน้าเว็บทันทีโดยไม่มีการ Reload หน้าเว็บ
- ฟอร์มเพิ่มรายการใหม่ (POST):** ดักจับอีเวนต์การส่งฟอร์มด้วย `e.preventDefault()` เพื่อป้องกันการรีเฟรชหน้าเว็บ แล้วส่งคำขอแบบ `POST` พร้อม JSON Body ไปยังเซิร์ฟเวอร์ เมื่อได้รับสถานะ `201 Created` จะรีเซ็ตฟอร์มและดึงข้อมูลใหม่มาแสดงผลทันที
- การลบรายการ (DELETE):** แต่ละรายการจะมีปุ่มลบที่ผูกกับฟังก์ชัน `deleteWorkout(id)` เมื่อผู้ใช้ยืนยันการลบ ไคลเอนต์จะส่งคำขอ `DELETE /api/workouts/:id` และอัปเดตหน้าจอทันทีเมื่อลบสำเร็จ
- การออกแบบ UI/CSS:** ออกแบบด้วย Responsive Clean UI ใช้การจัดวางสไตล์ Modern Card Layout ปรับแต่งระยะห่าง (Margin/Padding) และโทนสีเพื่อความสบายตาและใช้งานง่าย
- การสื่อสารข้อมูล:** เรียกใช้งานเฉพาะ REST API ภายในเซิร์ฟเวอร์ตัวเองเท่านั้น ไม่ได้มีการดึงข้อมูลจาก API ภายนอก

4. หลักฐานการดีบักระบบ (Debugging Evidence)

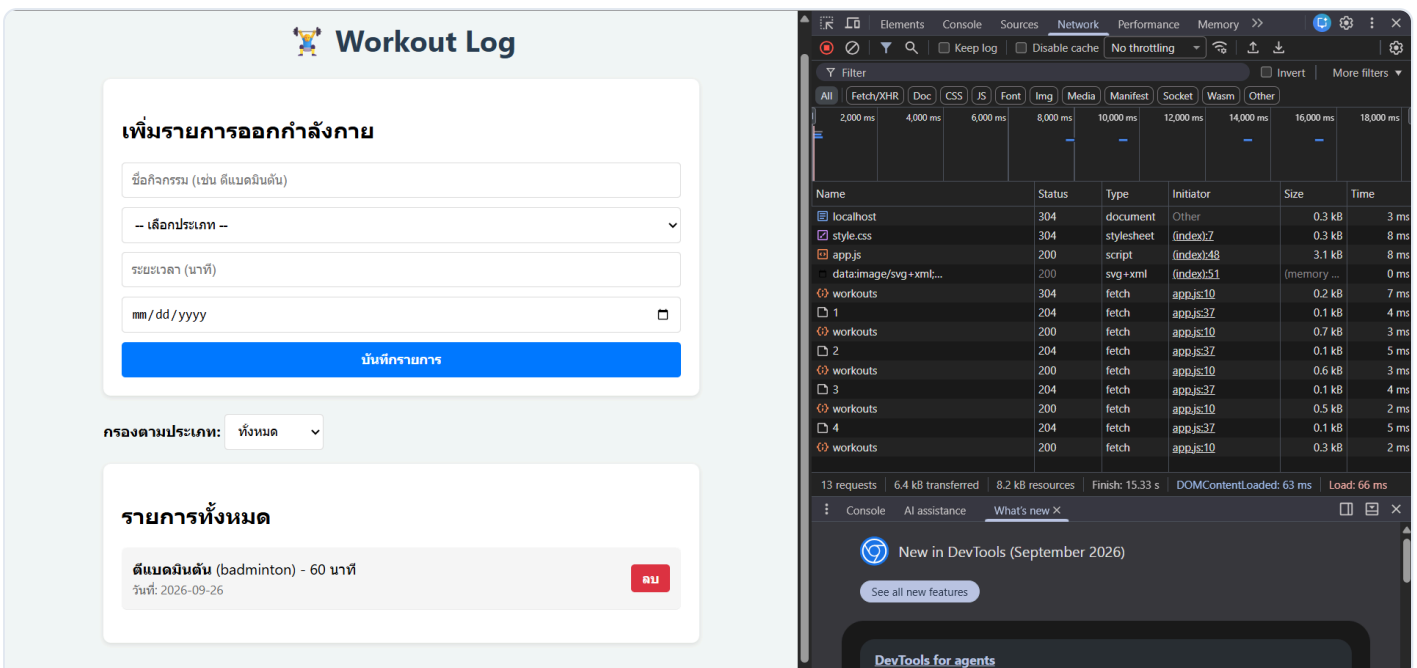
4.1 VS Code Debugger (Breakpoints)

ภาพแสดงการตั้งจุด Breakpoint ในไฟล์ `server/server.js` เพื่อตรวจสอบการทำงานของ Express Request Handler และการหยุดรอตรวจสอบค่าตัวแปรระหว่างรันโปรแกรม:



4.2 Chrome DevTools Network Tab

ภาพแสดงตาราง Network ใน Chrome DevTools ยืนยันการส่งคำขอ `POST` (Status 201), `GET` (Status 200) และ `DELETE` (Status 204) สำเร็จอย่างสมบูรณ์:



5. สรุปผลการดำเนินงาน

แอปพลิเคชัน Workout Log ผ่านการทดสอบและตรงตามข้อกำหนดของรายวิชาทุกประการ ทั้งการจัดโครงสร้างโปรเจกต์ การสร้าง RESTful API ครบถ้วนทุกคำสั่ง การประมวลผลฝั่งไคลเอนต์แบบ Asynchronous โดยไม่ reload หน้าจอ และการบันทึกเอกสารพร้อมรูปภาพหลักฐานบน GitHub Repository เรียบร้อยแล้ว